

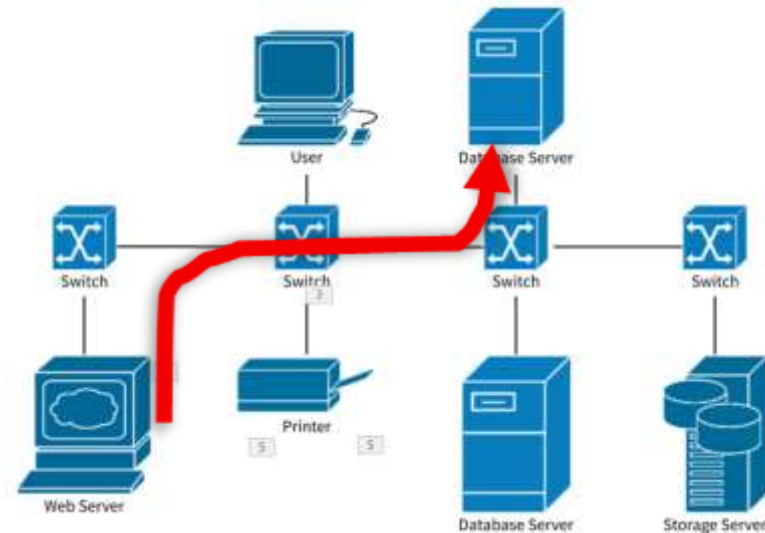
A photograph of a modern university building with large glass windows and a white facade, set against a blue sky with a few clouds. In the foreground, there is a field of tall grass and yellow wildflowers. A semi-transparent white box is overlaid on the image, containing the course title and the main topic.

5CM512 Communication and Security Protocols

Transport-Level Security

Web Security Considerations

- World Wide Web
 - fundamentally a client/server application running over the Internet and TCP/IP intranets
- Need for tailored security tools:
 - Web servers
 - Easy to configure and manage
 - Web content
 - Easy to develop
 - The underlying software
 - Extraordinarily complex
 - Hide many potential security flaws
 - Launching pad into entire computer complex
 - Clients
 - Casual and untrained (in security matters) users
 - Not necessarily aware of the security risks
 - Not have the tools or knowledge to take effective countermeasures



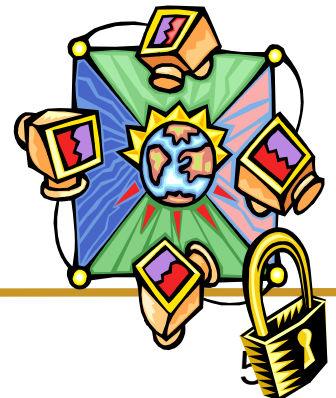
Class discussion

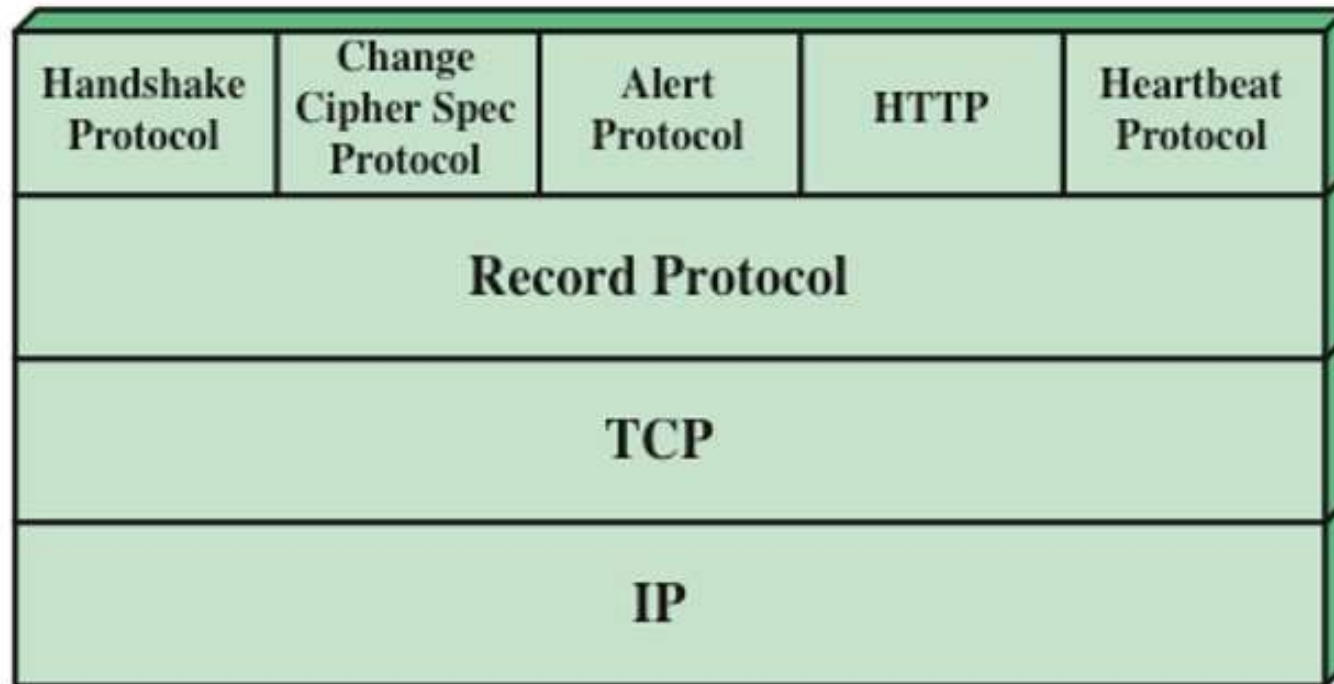
- Discuss possible threats for WEB Applications considering
 - Integrity
 - Confidentiality
 - Denial of Service
 - Authentication

	Threats	Consequences	Countermeasures
Integrity	•Modification of user data •Trojan horse browser •Modification of memory •Modification of message traffic in transit	•Loss of information •Compromise of machine •Vulnerability to all other threats	Cryptographic checksums
Confidentiality	•Eavesdropping on the net •Theft of info from server •Theft of data from client •Info about network configuration •Info about which client talks to server	•Loss of information •Loss of privacy	Encryption, Web proxies
Denial of Service	•Killing of user threads •Flooding machine with bogus requests •Filling up disk or memory •Isolating machine by DNS attacks	•Disruptive •Annoying •Prevent user from getting work done	Difficult to prevent
Authentication	•Impersonation of legitimate users •Data forgery	•Misrepresentation of user •Belief that false information is valid	Cryptographic techniques

Transport Layer security (TSL)

- One of the most widely used security services
- Internet standard
 - Evolved from a commercial protocol known as *Secure Sockets Layer* (SSL)
- General purpose service implemented as a set of protocols that rely on TCP
 - Part of the underlying protocol suite and therefore be transparent to applications
 - Embedded in specific packages





TLS Architecture

- Two important TLS concepts are:

TLS connection

- A transport that provides a suitable type of service
- For TLS such connections are peer-to-peer relationships
- Connections are transient
- Every connection is associated with one session

TLS session

- An association between a client and a server
- Created by the Handshake Protocol
- Define a set of cryptographic security parameters which can be shared among multiple connections
- Are used to avoid the expensive negotiation of new security parameters for each connection



A session state is defined by the following parameters:

Session identifier	→	An arbitrary byte sequence chosen by the server to identify an active or resumable session state
Peer certificate	→	An X509.v3 certificate of the peer; this element of the state may be null
Compression method	→	The algorithm used to compress data prior to encryption
Cipher spec	→	Specifies the bulk data encryption algorithm and a hash algorithm used for MAC calculation; also defines cryptographic attributes such as the hash size
Master secret	→	48-byte secret shared between the client and the server
Is resumable	→	A flag indicating whether the session can be used to initiate new connections



A connection state is defined by the following parameters:

Server and client random

- Byte sequences that are chosen by the server and client for each connection

Server write MAC secret

- The secret key used in MAC operations on data sent by the server

Client write MAC secret

- The secret key used in MAC operations on data sent by the client

Server write key

- The secret encryption key for data encrypted by the server and decrypted by the client

Client write key

- The symmetric encryption key for data encrypted by the client and decrypted by the server

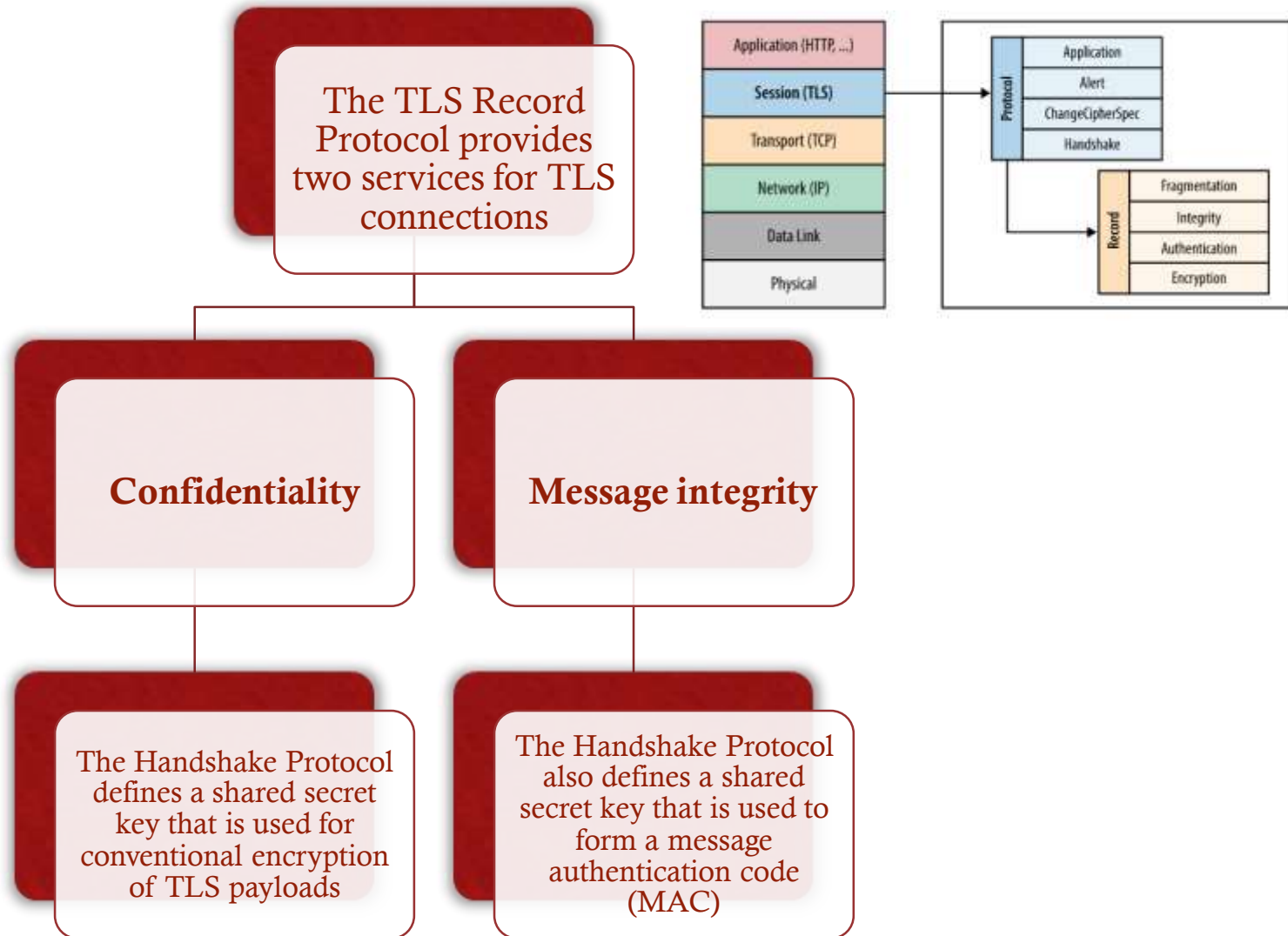
Initialization vectors

- When a block cipher in CBC mode is used, an initialization vector (IV) is maintained for each key
- This field is first initialized by the SSL Handshake Protocol
- The final ciphertext block from each record is preserved for use as the IV with the following record

Sequence numbers

- Each party maintains separate sequence numbers for transmitted and received messages for each connection
- When a party sends or receives a change cipher spec message, the appropriate sequence number is set to zero
- Sequence numbers may not exceed $2^{64} - 1$

TLS Record Protocol



Application Data

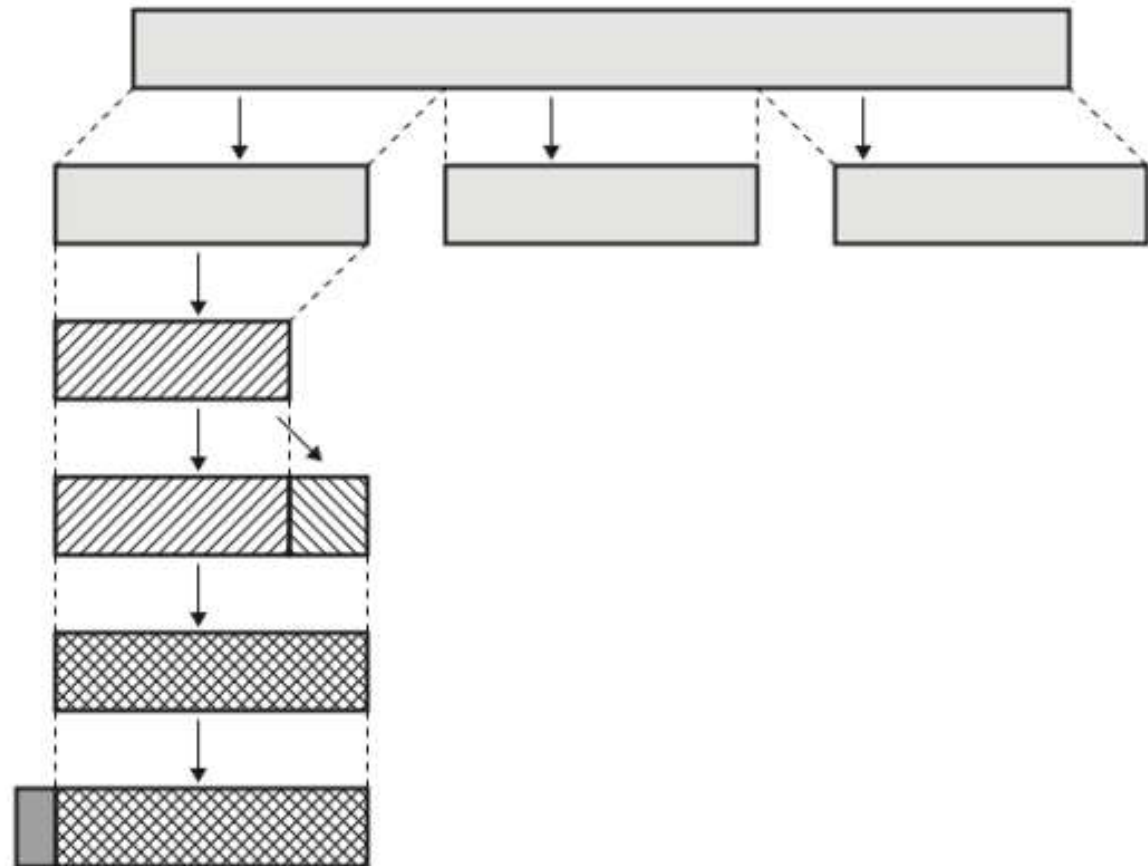
Fragment

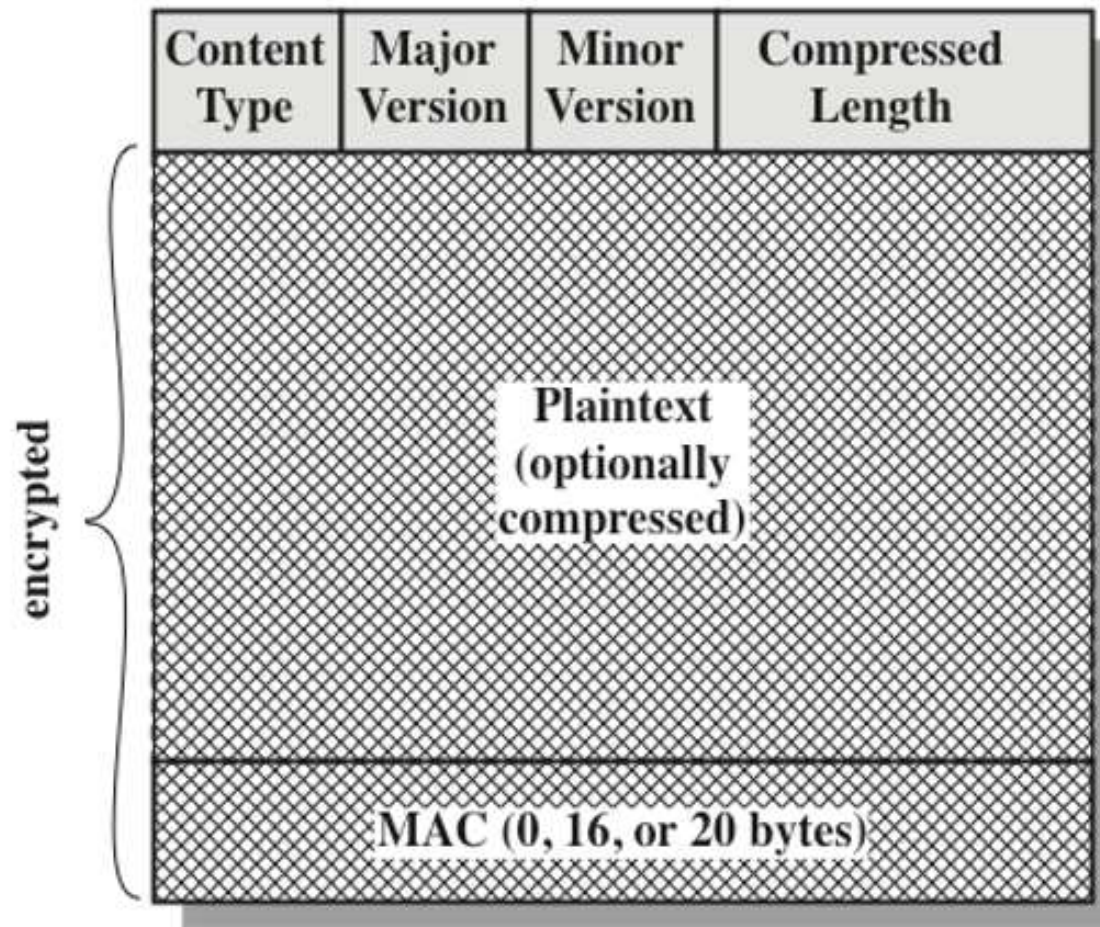
Compress

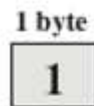
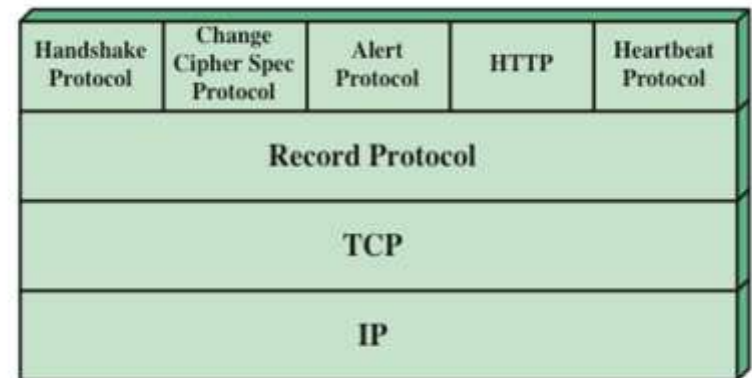
Add MAC

Encrypt

**Append TLS
Record Header**



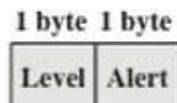




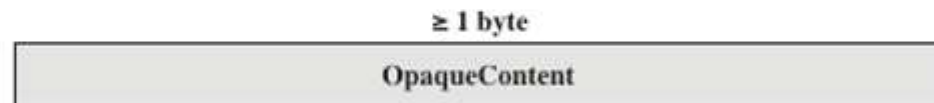
(a) Change Cipher Spec Protocol



(c) Handshake Protocol

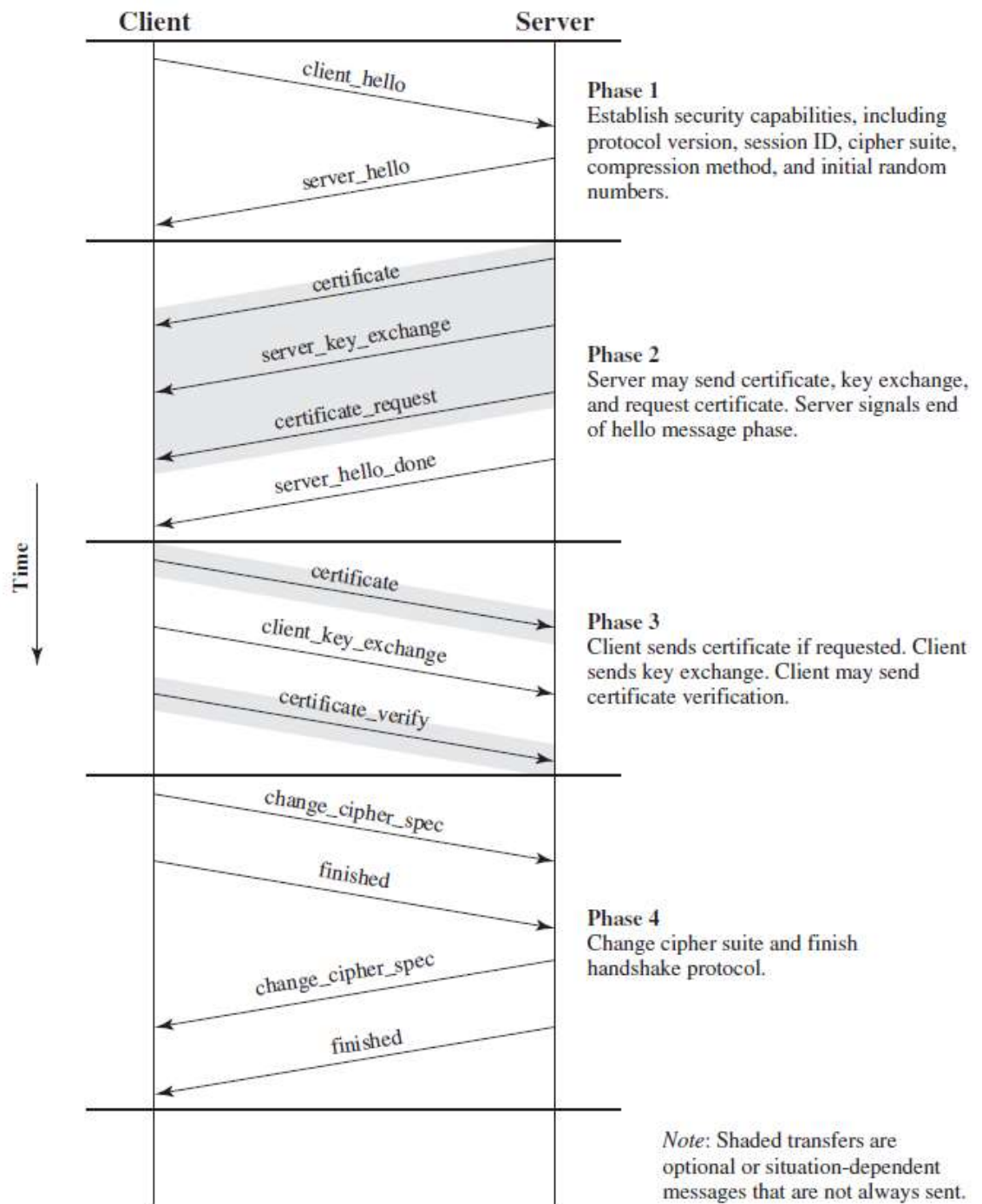


(b) Alert Protocol



(d) Other Upper-Layer Protocol (e.g., HTTP)

Message Type	Parameters
hello_request	null
client_hello	version, random, session id, cipher suite, compression method
server_hello	version, random, session id, cipher suite, compression method
certificate	chain of X.509v3 certificates
server_key_exchange	parameters, signature
certificate_request	type, authorities
server_done	null
certificate_verify	signature
client_key_exchange	parameters, signature
finished	hash value



Heartbeat Protocol

- Heartbeat
 - periodic signal generated by hardware or software
 - to indicate normal operation or to synchronize other parts of a system
- A heartbeat protocol
 - typically used to monitor the availability of a protocol entity
- Runs on top of the TLS Record Protocol
 - Consists of two message types
 - heartbeat_request
 - heartbeat_response
- The heartbeat serves two purposes:
 - It assures the sender that the recipient is still alive, even though there may not have been any activity over the underlying TCP connection
 - It generates activity across the connection during idle periods, which avoids closure by a firewall that does not tolerate idle connections



Any QUESTIONS?

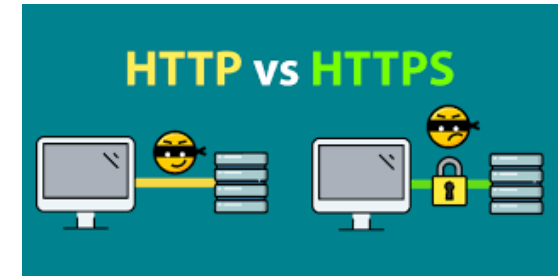


HTTPS (HTTP over SSL)

- Combination of HTTP and SSL



- Secure communication between a Web browser and a Web server
- Built into all modern Web browsers
 - URL addresses that begin with **https://** rather than **http://**



- HTTPS → port 443
 - invokes SSL



- Documented in RFC 2818, HTTP Over TLS
 - No fundamental change in using HTTP over either SSL or TLS
 - both implementations are referred to as HTTPS



Class discussion

- Why do we need additional security over HTTP?
- Do we need replace all HTTP based Web Sites to HTTPS?
- Can you give examples for web applications need HTTPS?

HTTPS (HTTP over SSL)

- Encrypted elements of communication:

- URL of the requested document



- Contents of the document



- Contents of browser forms



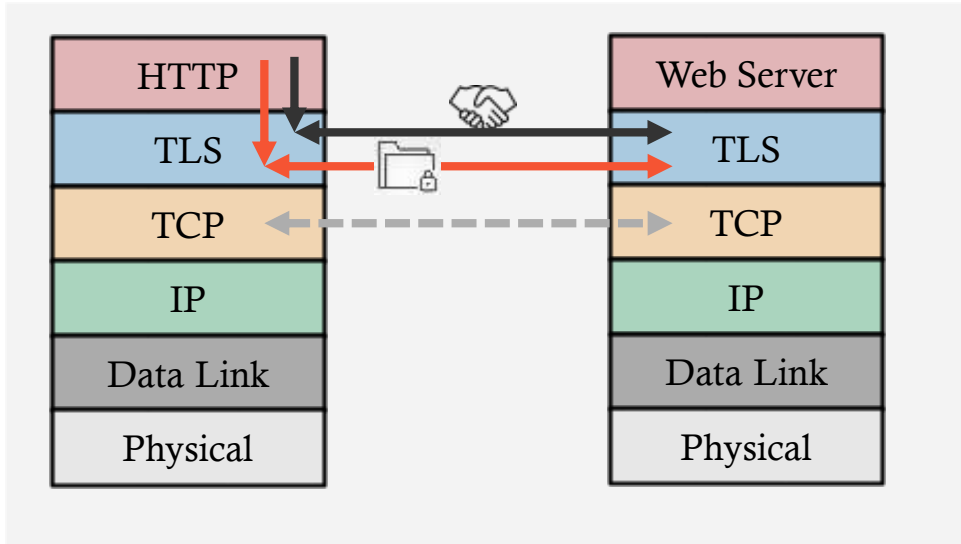
- Cookies sent from browser to server and from server to browser



- Contents of HTTP header



Connection Initiation



- HTTPS client
 - HTTP + TLS
 - Initiate a connection
- TLS
 - Handshake (session)
 - Connections
- HTTP
 - Send requests over TLS

Connection Closure

- An HTTP client or server can close a connection

- `Connection: close`



- Closure of an HTTPS connection

- TLS close the connection with the peer TLS entity on the remote side

- Closing the underlying TCP connection

- TLS implementations must initiate an exchange of closure alerts before closing a connection
 - A TLS implementation may, after sending a closure alert, close the connection without waiting for the peer to send its closure alert, generating an “incomplete close”
- An unannounced TCP closure could be evidence of some sort of attack so the HTTPS client should issue some sort of security warning when this occurs

Secure Shell (SSH)

A protocol for secure network communications

Simple and inexpensive to implement



SSH1

Providing a secure remote login facility

Replacing TELNET and other remote login schemes that provided no security



Provides a more general client/server capability

- Used for such network functions as file transfer and e-mail

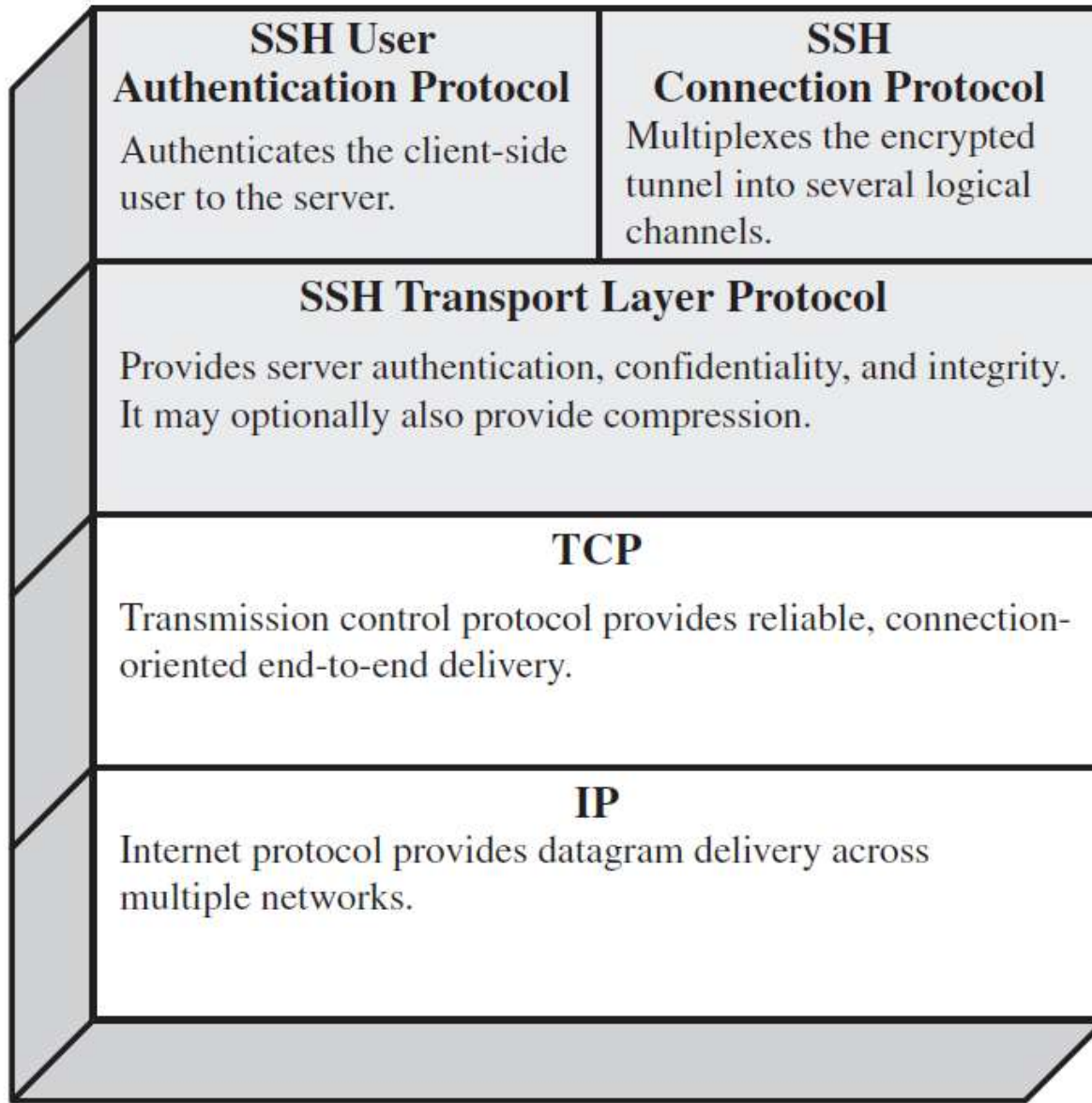
SSH client and server applications are widely available for most operating systems

- Has become the method of choice for remote login and X tunneling
- Is rapidly becoming one of the most pervasive applications for encryption technology outside of embedded systems

SSH2

fixes a number of security flaws



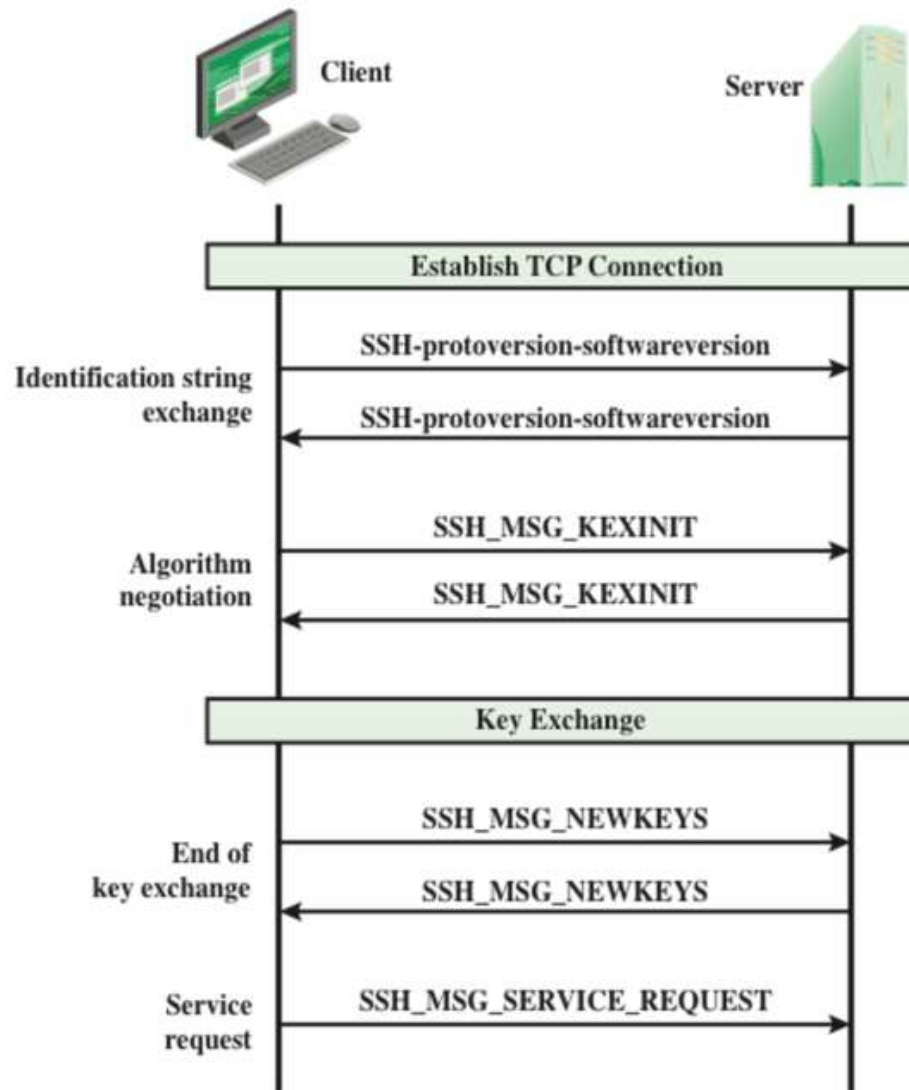


SSH Protocol Stack

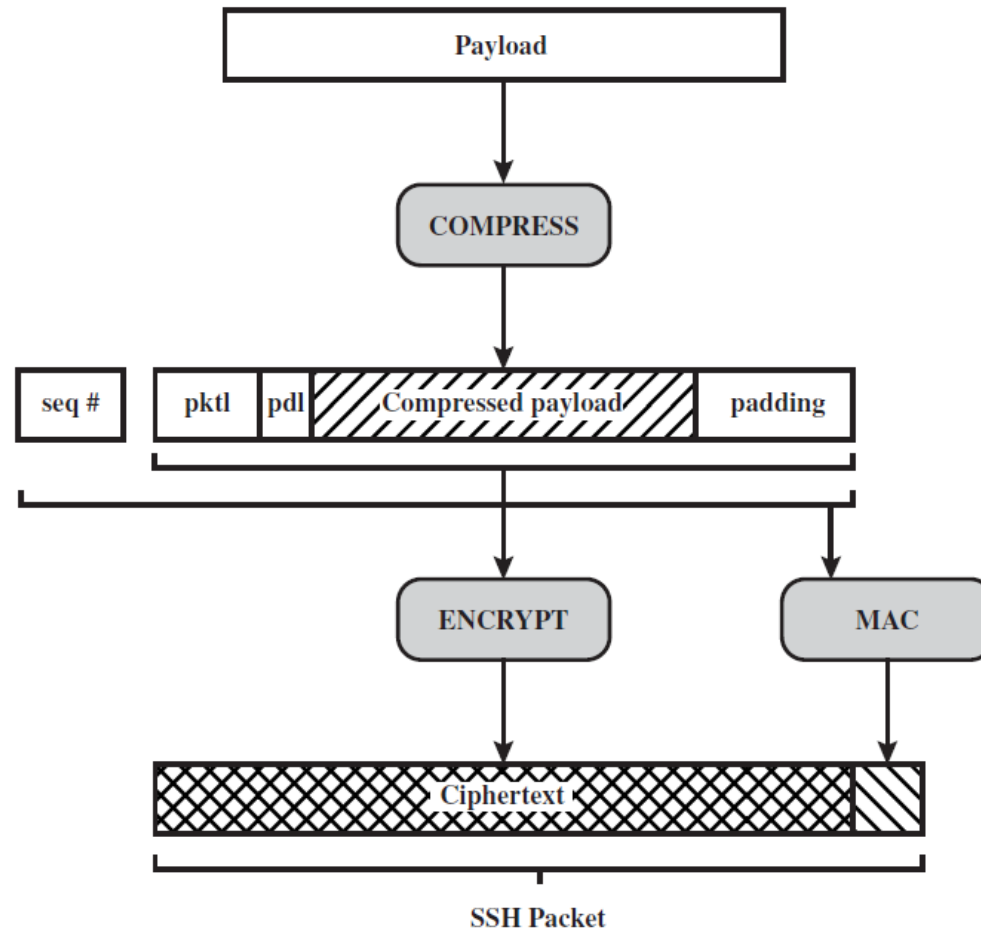
Transport Layer Protocol

- Server authentication occurs at the transport layer
 - based on the server possessing a public/private key pair
- Server
 - multiple host keys
 - using multiple different asymmetric encryption algorithms
- Multiple hosts may share the same host key
- The **server host key** is used during key exchange to authenticate the identity of the host
- Two alternative trust models:
 - The client has a local database that associates each host name with the corresponding public host key
 - The host name-to-key association is certified by a trusted certification authority (CA);
 - the client only knows the CA root key and can verify the validity of all host keys certified by accepted CAs

SSH Transport Layer Protocol Packet Exchanges



SSH Transport Layer Protocol Packet Formation



pktl = packet length
pdl = padding length

Cipher	
3des-cbc*	Three-key 3DES in CBC mode
blowfish-cbc	Blowfish in CBC mode
twofish256-cbc	Twofish in CBC mode with a 256-bit key
twofish192-cbc	Twofish with a 192-bit key
twofish128-cbc	Twofish with a 128-bit key
aes256-cbc	AES in CBC mode with a 256-bit key
aes192-cbc	AES with a 192-bit key
aes128-cbc**	AES with a 128-bit key
Serpent256-cbc	Serpent in CBC mode with a 256-bit key
Serpent192-cbc	Serpent with a 192-bit key
Serpent128-cbc	Serpent with a 128-bit key
arcfour	RC4 with a 128-bit key
cast128-cbc	CAST-128 in CBC mode

* = Required

** = Recommended

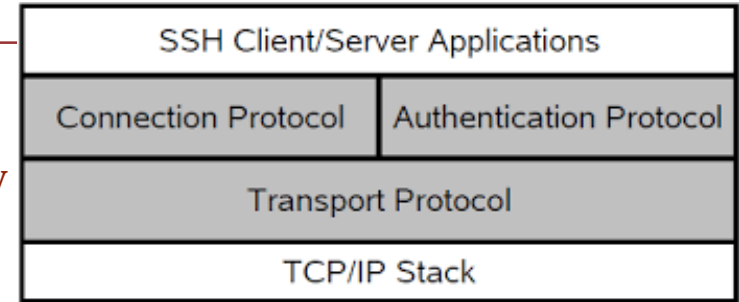
MAC algorithm	
hmac-sha1*	HMAC-SHA1; digest length = key length = 20
hmac-sha1-96**	First 96 bits of HMAC-SHA1; digest length = 12; key length = 20
hmac-md5	HMAC-SHA1; digest length = key length = 16
hmac-md5-96	First 96 bits of HMAC-SHA1; digest length = 12; key length = 16

Compression algorithm	
none*	No compression
zlib	Defined in RFC 1950 and RFC 1951

Authentication Methods

Publickey

- Client → Server :
 - Client's public key, Signed by the client's private key
- Server : Checks
 - Whether the key is acceptable for authentication
 - Whether the signature is correct



Password

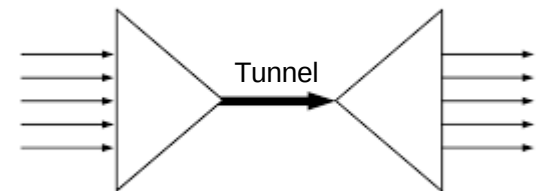
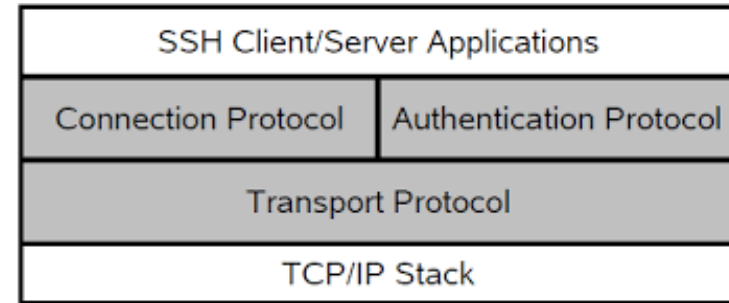
- Client → Server :
 - Plaintext password ; protected by encryption by the Transport Layer Protocol

Hostbased

- Authentication is performed on the client's host
- Client → Server :
 - Signature ; created with the private key of the client host
- Server :
 - verifies the identity of the client host

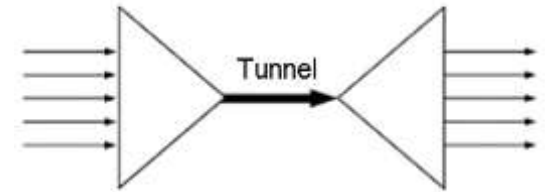
Connection Protocol

- The SSH Connection Protocol
 - runs on top of the SSH Transport Layer Protocol
 - assumes that a secure authentication connection is in use
 - The secure authentication connection
 - referred to as a *tunnel*
 - used by the Connection Protocol to multiplex a number of logical channels

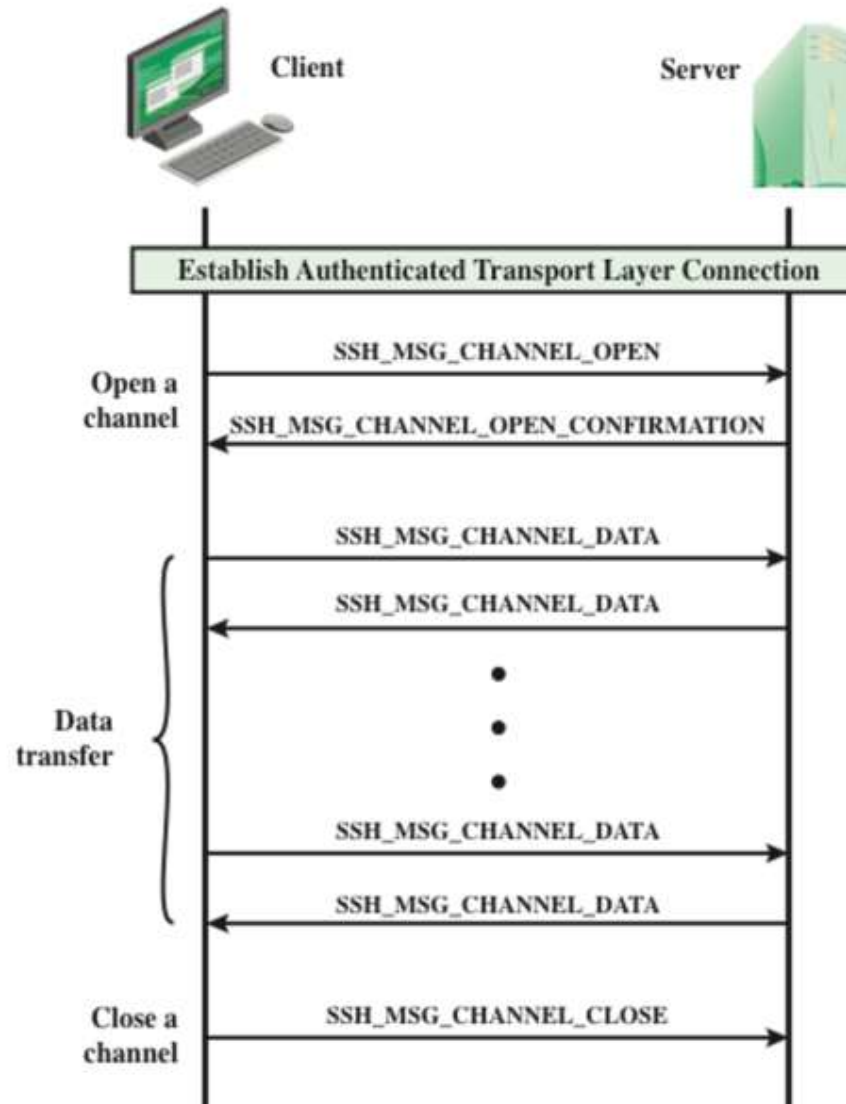


Connection Protocol

- Channel mechanism
 - All types of communication using SSH
 - supported using separate channels
 - Either side may open a channel
 - Unique channel number
 - Channels are flow controlled
 - using a window mechanism
 - No data may be sent to a channel until a message is received to indicate that window space is available
 - The life of a channel : Three stages
 - opening a channel,
 - data transfer
 - closing a channel



Example SSH Connection Protocol Message Exchange



Any QUESTIONS?

